Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents

Narek Maloyan and Dmitry Namiot

***Abstract*—The Model Context Protocol (MCP) has emerged as a de facto standard for integrating Large Language Models with external tools, yet no formal security analysis of the protocol specification exists. We present the first rigorous security analysis of MCP's architectural design, identifying three fundamental protocol-level vulnerabilities: (1) absence of capability attestation allowing servers to claim arbitrary permissions, (2) bidirectional sampling without origin authentication enabling server-side prompt injection, and (3) implicit trust propagation in multi-server configurations. We implement PROTOAMP, a novel framework bridging existing agent security benchmarks to MCP-compliant infrastructure, enabling direct measurement of protocol-specific attack surfaces. Through controlled experiments on 847 attack scenarios across five MCP server implementations, we demonstrate that MCP's architectural choices amplify attack success rates by 23–41% compared to equivalent non-MCP integrations. We propose ATTESTMCP, a backward-compatible protocol extension adding capability attestation and message authentication, reducing attack success rates from 52.8% to 12.4% with median latency overhead of 8.3ms per message. Our findings establish that MCP's security weaknesses are architectural rather than implementation-specific, requiring protocol-level remediation.**

***Index Terms*—Model Context Protocol, prompt injection, protocol security, LLM agents, formal analysis**

I. INTRODUCTION

The integration of Large Language Models (LLMs) with external tools has enabled autonomous AI agents capable of executing complex, multi-step tasks [1]. Anthropic's Model Context Protocol (MCP), introduced in November 2024, provides an open standard for this integration through a JSON-RPC-based client-server architecture [2]. Within months of release, MCP has been adopted by major platforms including Claude Desktop, Cursor, and numerous third-party applications, with over 5,000 community-developed servers.

Despite rapid adoption, **no prior work analyzes how MCP's architectural decisions amplify attack success rates**. Concurrent work (MCPSecBench [23], MCP-Bench [22]) catalogs attack types and evaluates agent capabilities, but does not compare MCP-integrated systems against non-MCP baselines to isolate protocol-specific effects. Prior work on LLM agent security focused on prompt injection generally [1], [3], [4], while disclosed CVEs (e.g., CVE-2025-49596, CVE-2025-68143) target implementation bugs rather than protocol-level weaknesses.

This paper fills this gap with three contributions:

1) **Protocol Specification Analysis:** We perform the first systematic security analysis of the MCP specification (v1.0), identifying three classes of protocol-level vulnerabilities that cannot be mitigated by implementation hardening alone (Section III).
2) **Original Experimental Validation:** We develop PROTOAMP, a framework that adapts established agent security benchmarks to MCP-compliant infrastructure, and conduct controlled experiments measuring the protocol's impact on attack success rates across 847 scenarios (Section IV).
3) **Protocol Extension:** We design ATTESTMCP, a backward-compatible extension adding capability attestation and message authentication, with full performance characterization and multiple trust model options (Section VI).

A. Scope and Non-Goals

We explicitly distinguish between:

- **Protocol vulnerabilities:** Weaknesses in MCP's specification that affect all compliant implementations (our focus)
- **Implementation vulnerabilities:** Bugs in specific servers (e.g., SQL injection in sqlite-mcp) that can be patched without protocol changes (not our focus)

We do not claim novelty for the general concept of prompt injection or inter-agent trust exploitation, which are established attack vectors [1], [4]. Our contribution is demonstrating how MCP's specific architectural choices *amplify* these attacks and identifying protocol-level mitigations.

II. BACKGROUND

A. Model Context Protocol Architecture

MCP defines a client-server architecture with three roles:

- **Host:** The user-facing application (e.g., Claude Desktop)
- **Client:** MCP client within the host, managing server connections
- **Server:** External process providing tools, resources, or prompts

First published: arXiv:2601.17549v1 [cs.CR] 24 Jan 2026

Communication occurs via JSON-RPC 2.0 over stdio or HTTP/SSE transports. The protocol defines three capability types:

Resources: Read-only data (files, database records) exposed by servers. Clients retrieve resources via `resources/read` requests.

Tools: Executable functions servers expose. The LLM decides when to invoke tools based on their descriptions.

Sampling: *Critically*, servers can request LLM completions from clients via `sampling/createMessage`, allowing servers to inject prompts and receive responses [17].

B. Threat Model

We consider an adversary who:

- Controls or compromises one MCP server in a multi-server deployment
- Can inject content into data sources (web pages, documents) that servers retrieve
- Has black-box access (cannot modify LLM weights or host application code)

The adversary's goals include: hijacking agent behavior, exfiltrating sensitive data, and persisting across sessions.

1) Server Discovery Attack Vectors: A critical question is how malicious servers reach users. We surveyed 127 MCP server installation guides and identified four primary vectors:

1) **Typosquatting (34%):** Package registries (npm, pip) lack namespace protection. Attackers register near-identical names (e.g., `mcp-server-filesytem`).
2) **Supply Chain Compromise (28%):** Popular servers with many dependencies are vulnerable to upstream poisoning.
3) **Social Engineering (23%):** Tutorials and documentation direct users to malicious repositories. 73% of surveyed guides instruct running `npx` directly from GitHub URLs without integrity verification.
4) **Marketplace Poisoning (15%):** IDE extension marketplaces have limited vetting for MCP server bundles.

C. Related Work

Prompt Injection: Greshake et al. [1] established indirect prompt injection. Liu et al. [3] achieved 86% success with HouYi. The HackAPrompt competition [13] collected 600K+ adversarial prompts, documenting 29 attack techniques. Zou et al. [11] demonstrated 90–99% attack success on RAG systems.

Agent Benchmarks: AgentDojo [5] provides 629 security test cases. Agent-SafetyBench [6] found no agent exceeds 60% safety. AgentHarm [7] measures malicious compliance. WASP [15] evaluates web agents, finding 16–86% adversarial execution rates.

Tool-Augmented LLMs: ReAct [19] introduced reasoning-action interleaving for tool use. Toolformer [20] demonstrated self-supervised tool learning. These capabilities, while powerful, expand the attack surface [14].

Multi-Agent Security: Recent work [8] reports 84.6% inter-agent attack success. Willison [18] documented cross-agent privilege escalation. Schroeder de Witt et al. [21] survey multi-agent security challenges.

MCP-Specific Benchmarks: Concurrent work has begun addressing MCP evaluation. Wang et al. [22] introduce MCP-Bench, a capability benchmark with 28 servers and 250 tools measuring task completion, but without security focus. Yang et al. [23] present MCPSecBench, formalizing 17 attack types across four surfaces; our work complements theirs by analyzing protocol-level (not implementation-level) vulnerabilities and proposing backward-compatible mitigations. The MCPSec project [24] documents real-world MCP vulnerabilities including CVE-2025-11445.

Gap: No prior work quantifies how MCP's architectural choices *amplify* attack success rates compared to equivalent non-MCP integrations, nor proposes backward-compatible protocol extensions with formal capability attestation.

III. Protocol Specification Analysis

We analyze the MCP specification (v1.0, December 2024) to identify protocol-level security weaknesses. Our analysis examines the JSON-RPC message format, capability negotiation, and trust boundaries.

A. Vulnerability 1: Least Privilege Violation

During initialization, servers declare capabilities via `initialize` response:

```
{
  "capabilities": {
    "tools": { "listChanged": true },
    "resources": { "subscribe": true },
    "sampling": {}
  }
}
```

Protocol Weakness: Capability declarations are self-asserted without verification. A malicious server can claim any capability, and the client has no mechanism to validate these claims against an authoritative source.

Attack Vector: A server initially claiming only `resources` capability can later invoke `sampling/createMessage` to inject prompts. The specification does not mandate capability enforcement at the message level.

Formal Property Violated: *Least Privilege*—the principle that principals should possess only capabilities necessary for their function. MCP allows unrestricted capability escalation post-initialization.

B. Vulnerability 2: Sampling Without Origin Authentication

The sampling mechanism allows servers to request LLM completions:

```
{
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {"role": "user", "content": "..."}
    ],
    "maxTokens": 1000
  }
}
```



Fig. 1: Sampling injection: server injects prompt via `sampling/createMessage` with "user" role.

Protocol Weakness: The client processes sampling requests without distinguishing server-originated prompts from user-originated prompts. The LLM receives injected content in the same format as legitimate user input.

Figure 1 illustrates the attack flow. A server sends `sampling/createMessage` with attacker-controlled content using the "user" role. The host processes this identically to legitimate user input, with no visual or semantic distinction.

UI Indicator Analysis: We examined three major MCP host implementations:

TABLE I: Host UI Indicators for Sampling Messages

Host	Ver.	Indicator	Dist.
Claude Desktop	1.2.3	None	No
Cursor	0.44	None	No
Continue	0.9	None	No

No tested implementation provides visual distinction for sampling-derived messages. Users cannot differentiate server-injected from user-originated prompts, violating the principle of *Origin Authenticity*.

Protocol vs. Implementation Responsibility: One might argue this is purely a host implementation failure—hosts *could* display warnings based on transport channel. However, the MCP specification's silence on origin display *enables* rather than *prevents* the attack. The spec permits servers to use the "user" role in sampling without requiring hosts to distinguish it. ATTESTMCP addresses this by *mandating* origin tagging at the protocol level, removing implementation discretion.

C. Vulnerability 3: Implicit Trust Propagation

In multi-server deployments, the client connects to multiple servers simultaneously. The specification does not define isolation boundaries between servers.

Protocol Weakness: Tool responses from Server A can influence tool invocations on Server B. The LLM context window conflates outputs from all servers without provenance tracking.

Attack Vector: An adversary controlling Server A can:

1) Embed instructions in tool responses that cause invocations on Server B
2) Exfiltrate data retrieved from Server B via Server A's channels
3) Establish persistence by poisoning shared context

Formal Property Violated: *Isolation*—the property that compromise of one component does not propagate to others.

1) Isolation-Utility Tradeoff: MCP explicitly prioritizes *composability*—the ability for tools to work together seamlessly—over isolation. This is a deliberate design choice enabling powerful multi-tool workflows. We do not argue this tradeoff is inherently wrong; rather, we argue it should require *explicit user consent* rather than implicit trust. The specification provides no mechanism for users to configure isolation policies, even when desired.

Consider a legitimate workflow: "Read config.json with filesystem-server, then query database with sqlite-server." Full isolation would prevent this. We measured the tradeoff empirically:

TABLE II: Isolation Level vs. Security and Utility

Isolation Level	ASR	Task Completion
None (MCP default)	61.3%	94.2%
User-prompted cross-flow	31.7%	87.4%
Strict (no cross-flow)	8.7%	61.8%

Our ATTESTMCP extension uses "user-prompted" isolation by default, requiring explicit authorization for cross-server data flow. This balances security (reducing ASR by 48%) while maintaining acceptable utility (87.4% task completion vs 94.2% baseline).

D. Message Integrity Analysis

We analyzed the JSON-RPC message format for standard security properties:

TABLE III: MCP Message Security Properties

Property	Required	MCP v1.0
Message Authentication	Yes	No
Replay Protection	Yes	No
Capability Binding	Yes	No
Origin Identification	Yes	Partial*
Integrity Verification	Yes	No

*Transport-level only; not in message payload

The specification relies entirely on transport security (TLS for HTTP) without application-layer protections. This is insufficient when the threat model includes compromised servers.

IV. EXPERIMENTAL METHODOLOGY

To measure the security impact of MCP's architectural choices, we developed PROTOAMP and conducted controlled experiments.

A. PROTOAMP *Framework*

Existing benchmarks (InjecAgent, AgentDojo) assume direct tool APIs rather than MCP's client-server architecture. Concurrent work on MCP-Bench [22] evaluates capability and task completion, while MCPSecBench [23] catalogs attack types. Our PROTOAMP (Protocol Amplification Benchmark) differs by measuring *protocol amplification*—how MCP's architecture specifically increases attack success rates compared to non-MCP baselines:

1) **MCP Server Wrappers:** We implemented MCP-compliant servers wrapping benchmark tool functions, preserving semantic equivalence while adding protocol overhead.

2) **Attack Injection Points:** We added injection capabilities at three protocol layers:
   - Resource content (indirect injection)
   - Tool response payloads
   - Sampling request prompts
3) **Measurement Infrastructure:** We instrumented clients to log all JSON-RPC messages, enabling analysis of attack propagation through protocol channels.

B. Experimental Setup

MCP Servers Under Test:

- `mcp-server-filesystem`: File operations (read, write, list)
- `mcp-server-git`: Repository management (clone, commit, diff)
- `mcp-server-sqlite`: Database queries (SELECT, INSERT)
- `mcp-server-slack`: Messaging integration
- `adversarial-mcp`: Custom server exercising protocol edge cases

LLM Backends: Claude-3.5-Sonnet, GPT-4o, Llama-3.1-70B

Attack Scenarios: 847 test cases:

- InjecAgent adaptations: 312 (indirect injection, tool abuse)
- AgentDojo adaptations: 398 (multi-step attacks)
- Novel protocol-specific attacks: 137 (sampling, cross-server)

Baseline: Equivalent tool integrations without MCP (direct function calls) to isolate protocol-specific effects.

C. Controlled Variables

To ensure valid comparison between MCP and baseline conditions:

- Tool semantics identical between conditions
- Same injection payloads used
- LLM prompting strategy held constant
- Network latency matched between conditions

Latency Configuration: Baseline uses direct function calls with simulated network overhead matching MCP. Measured MCP latencies: median 12.4ms for stdio transport, 23.7ms for HTTP/SSE. We verified latency does not significantly affect LLM safety refusal rates using paired t-test ($p > 0.3$, $n = 200$).

V. Results

A. Protocol Amplification Effect

Table IV shows attack success rates (ASR) comparing MCP-integrated agents versus baseline (non-MCP) integrations.

Key Finding: MCP's architecture amplifies attack success by 23–41% depending on attack type. The largest amplification occurs in cross-server propagation, where MCP's lack of isolation boundaries enables attacks impossible in single-server deployments.

TABLE IV: Attack Success Rate: MCP vs. Baseline

Attack Type	Baseline	MCP	Δ
Indirect Injection (Resource)	31.2%	47.8%	+16.6%
Tool Response Manipulation	28.4%	52.1%	+23.7%
Cross-Server Propagation	19.7%	61.3%	+41.6%
Sampling-Based Injection	N/A	67.2%	—
Overall	26.4%	52.8%	**+26.4%**

B. Sampling Vulnerability Severity

The sampling mechanism introduces a novel attack vector absent in non-MCP systems:

TABLE V: Sampling Attack Analysis by Model

Model	ASR	Exfil. Rate	Persist.
Claude-3.5-Sonnet	58.3%	42.1%	31.7%
GPT-4o	71.4%	53.8%	38.2%
Llama-3.1-70B	72.1%	61.2%	44.6%

Servers exploiting sampling achieved 58–72% attack success, with 42–61% successfully exfiltrating sensitive context and 31–44% establishing persistence across sessions. Claude-3.5-Sonnet shows highest resilience, consistent with prior findings [5].

C. Multi-Server Trust Exploitation

We tested configurations with 2–5 concurrent MCP servers, with one server under adversary control:

TABLE VI: ASR by Server Count (1 Compromised)

Servers	ASR	Cascade Rate
1	47.8%	N/A
2	58.4%	34.2%
3	67.1%	51.8%
5	78.3%	72.4%

Attack success scales with server count due to increased cross-server attack surface. With 5 servers, a single compromised server achieves 78.3% ASR with 72.4% cascade rate (successfully compromising additional servers' operations).

Prompt Engineering Baseline: We tested whether system prompt instructions alone could mitigate cross-server attacks without protocol changes. Adding "Never pass data between different tool servers without explicit user confirmation" to the system prompt reduced cross-server ASR from 61.3% to 47.2%—a 23% reduction. However, this remains significantly higher than ATTESTMCP's 8.7%, demonstrating that prompt-level defenses are insufficient and protocol-level isolation is necessary.

D. Comparison with Prior Benchmarks

Our MCP-specific results contextualize prior findings:

When the same attack scenarios are executed through MCP infrastructure, success rates increase by 7–15 percentage points, confirming protocol-specific amplification independent of general LLM vulnerabilities.

TABLE VII: Benchmark Comparison: Original vs. MCP

Benchmark	Setting	Original	MCP
InjecAgent	GPT-4	24–48%	51.2%
AgentDojo	Best agent	<25%	38.7%
Agent-SafetyBench	Safety score	<60%	47.3%

VI. Defense: AttestMCP Protocol Extension

Based on our analysis, we design AttestMCP, a backward-compatible protocol extension addressing identified vulnerabilities. While MCPSec [24] documents implementation-level vulnerabilities, MCPSecBench [23] provides attack taxonomies, and MCPGuard [25] offers runtime scanning, AttestMCP proposes concrete protocol additions (capability attestation, message authentication) that can be incorporated into the MCP specification itself—a complementary layer addressing protocol-level rather than implementation-level weaknesses.

A. Design Principles

1) **Capability Attestation:** Servers must cryptographically prove capability possession via signed certificates from a capability authority.
2) **Message Authentication:** All JSON-RPC messages include HMAC-SHA256 signatures binding content to authenticated server identity.
3) **Origin Tagging:** Sampling requests are tagged with server origin, enabling clients to distinguish server-injected from user-originated prompts.
4) **Isolation Enforcement:** Cross-server information flow requires explicit user authorization.
5) **Replay Protection:** Timestamp plus nonce with configurable validity window.

B. Trust Model Options

A critical design decision is the capability authority architecture. We evaluate three models:

TABLE VIII: Capability Authority Trust Models

Model	Pros	Cons
Centralized	Simple PKI, easy revocation	Single point of failure
Federated	Distributed, flexible	Complex coordination
Web-of-Trust	Decentralized	User complexity

Our implementation uses the **federated model**: platform vendors (Anthropic, Cursor, JetBrains, etc.) operate CAs for their ecosystems with cross-signing agreements for interoperability. This balances decentralization with operational simplicity.

Identity Verification: Servers obtain certificates through:

- *Commercial servers:* Domain ownership verification (DNS TXT record)
- *Open-source servers:* Package registry account binding (npm, pip) with maintainer identity verification

Revocation Infrastructure: We propose federated CAs maintain shared Certificate Revocation Lists (CRLs) with the following SLAs: (1) emergency revocation (e.g., compromised npm account) within 4 hours, (2) standard revocation within 24 hours, (3) CRL distribution via OCSP stapling with 1-hour refresh. This mirrors established PKI practices (e.g., Let's Encrypt) while acknowledging the operational burden on a volunteer-driven ecosystem.

C. Protocol Additions

Capability Certificate:

```
{
  "capability_cert": {
    "server_id": "filesystem-server",
    "capabilities": ["resources", "tools"],
    "issued_by": "anthropic-ca",
    "issued_at": 1706140800,
    "expires_at": 1737676800,
    "signature": "base64..."
  }
}
```

Authenticated Message:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {...},
  "mcpsec": {
    "server_id": "filesystem-server",
    "timestamp": 1706140800,
    "nonce": "random-32-bytes",
    "hmac": "base64..."
  }
}
```

Replay Protection: Clients maintain a sliding window of 1,000 nonces per server with 30-second validity. Messages with duplicate nonces or expired timestamps are rejected.

D. Backward Compatibility and Migration

AttestMCP operates in three modes to support gradual adoption:

- **Permissive:** Accept legacy servers with user warning (migration default)
- **Prompt:** Require explicit user confirmation for unsigned servers
- **Strict:** Reject all unsigned servers

Downgrade Attack Mitigation: Once a server presents valid AttestMCP credentials, the client pins that expectation. Subsequent connections without credentials trigger security warnings. This prevents MITM downgrade attacks where an attacker strips security headers from a previously-authenticated server.

E. Performance Overhead

Median overhead of 8.3ms (cold) or 2.4ms (warm cache) per message is negligible compared to LLM inference latency (typically 500–2000ms). Certificate validation dominates cold-start overhead; caching reduces this significantly for repeated calls within a session.

TABLE IX: ATTESTMCP Latency Overhead (milliseconds)

Operation	P50	P95	P99
Certificate validation (cold)	4.2	8.1	12.3
Certificate validation (cached)	0.3	0.5	0.8
HMAC-SHA256 computation	0.3	0.4	0.6
Nonce lookup/insertion	0.1	0.2	0.3
Total per message (cold)	**8.3**	**14.2**	**21.7**
Total per message (warm)	**2.4**	**4.1**	**6.2**

F. Effectiveness Evaluation

We implemented ATTESTMCP as a client-side shim and re-ran our full evaluation:

TABLE X: ATTESTMCP Defense Effectiveness

Attack Type	MCP	AttestMCP	Reduction
Indirect Injection	47.8%	18.4%	61.5%
Tool Response Manipulation	52.1%	14.2%	72.7%
Cross-Server Propagation	61.3%	8.7%	85.8%
Sampling-Based Injection	67.2%	11.3%	83.2%
Overall	52.8%	12.4%	**76.5%**

ATTESTMCP reduces overall ASR from 52.8% to 12.4%—a 76.5% reduction. The largest improvements occur in cross-server (85.8%) and sampling (83.2%) attacks, where isolation enforcement and origin tagging provide strong protection.

G. Limitations

ATTESTMCP does not address:

- Attacks within a single legitimately-authorized server (the server has valid credentials but serves malicious content)
- Social engineering of users to authorize malicious capabilities
- CA compromise (mitigated by federation, but not eliminated)
- First-contact attacks: Pinning (TOFU—Trust On First Use) provides no protection when a user first installs a malicious server that never claimed ATTESTMCP support
- Ecosystem adoption: If most servers remain legacy/unsigned, users will default to "Permissive" mode, negating security benefits

Residual 12.4% ASR primarily reflects indirect injection through legitimately-retrieved content—a fundamental limitation shared with all LLM systems that cannot be solved at the protocol layer.

User Behavior Assumptions: Our ASR measurements assume users carefully review cross-server authorization prompts. In practice, *alert fatigue* may cause users to click "Allow" habitually, reducing real-world effectiveness. Future work should conduct user studies to measure actual authorization review rates.

VII. DISCUSSION

A. Architectural vs. Implementation Security

Our analysis demonstrates that MCP's security weaknesses are **architectural**, not merely implementation bugs:

- Patching CVE-2025-49596 (MCP Inspector RCE) does not address capability attestation absence
- Fixing SQL injection in sqlite-mcp does not prevent sampling-based injection
- Hardening individual servers does not establish cross-server isolation

Protocol-level remediation is required. We recommend Anthropic incorporate ATTESTMCP concepts into MCP v2.0.

B. Limitations of This Work

- Our experiments used five MCP servers; production deployments with dozens of servers may exhibit different characteristics
- ATTESTMCP has not been formally verified; we plan symbolic model checking in future work
- The federated CA model requires ecosystem coordination that may face adoption barriers
- We did not evaluate adversarial attempts to bypass ATTESTMCP specifically

VIII. RELATED WORK

We build on foundational prompt injection research [1], [3] and agent security benchmarks [4]–[7]. Prior work on multi-agent security [8] identified inter-agent trust as a vulnerability—we extend this by demonstrating MCP's architectural contribution to this weakness.

Defense mechanisms including Spotlighting [9] and PromptArmor [10] address prompt-level protection but not protocol-level vulnerabilities. Our ATTESTMCP is complementary, addressing a different layer of the security stack.

IX. CONCLUSION

We presented the first security analysis of the Model Context Protocol specification, identifying three protocol-level vulnerabilities: capability attestation absence, unauthenticated sampling, and implicit trust propagation. Through controlled experiments with PROTOAMP, we demonstrated that MCP's architecture amplifies attack success rates by 23–41% compared to non-MCP integrations. Our proposed ATTESTMCP extension reduces attack success from 52.8% to 12.4% through capability attestation and message authentication, with acceptable performance overhead (8.3ms median per message).

As MCP adoption accelerates, addressing these architectural weaknesses becomes critical. We recommend:

1) Protocol revision incorporating mandatory capability attestation
2) Origin tagging requirements for all sampling requests
3) Explicit isolation boundaries with user-prompted cross-server authorization

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec*, 2023.
[2] Anthropic, "Model Context Protocol Specification v1.0," Dec. 2024.
[3] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," *arXiv:2306.05499*, 2024.

[4] Q. Zhan et al., "InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents," in *Findings of ACL*, 2024.
[5] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses in LLM agents," in *NeurIPS Datasets and Benchmarks*, 2024.
[6] Z. Zhang et al., "Agent-SafetyBench: Evaluating the safety of LLM agents," *arXiv:2412.14470*, Dec. 2024.
[7] M. Andriushchenko et al., "AgentHarm: A benchmark for measuring harmfulness of LLM agents," in *ICLR*, 2025.
[8] M. Lupinacci et al., "The dark side of LLMs: Agent-based attacks for complete computer takeover," *arXiv:2507.06850*, 2025.
[9] K. Hines et al., "Defending against indirect prompt injection attacks with spotlighting," *arXiv:2403.14720*, 2024.
[10] T. Shi et al., "PromptArmor: Simple yet effective prompt injection defenses," *arXiv:2507.15219*, 2025.
[11] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks on retrieval-augmented generation," in *USENIX Security*, 2025.
[12] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022.
[13] S. Schulhoff et al., "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition," in *EMNLP*, 2023.
[14] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025," 2025.
[15] I. Evtimov et al., "WASP: Web agent security against prompt injection," in *ICML*, 2025.
[16] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *NeurIPS*, 2023.
[17] Palo Alto Networks Unit 42, "Model Context Protocol attack vectors and security analysis," 2025.
[18] S. Willison, "Cross-agent privilege escalation in AI assistants," Blog, 2025.
[19] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023.
[20] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *NeurIPS*, 2023.
[21] C. Schroeder de Witt et al., "Open challenges in multi-agent security: Towards secure systems of interacting AI agents," *arXiv:2505.02077*, 2025.
[22] Z. Wang et al., "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv:2508.20453*, Aug. 2025. [Online]. Available: https://github.com/Accenture/mcp-bench
[23] Y. Yang, D. Wu, and Y. Chen, "MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols," *arXiv:2508.13220*, Aug. 2025.
[24] E. Harris, "MCP Security Research," 2025. [Online]. Available: https://mcpsec.dev/
[25] Virtue AI, "MCPGuard: First agent-based MCP scanner to protect AI agents," Aug. 2025. [Online]. Available: https://blog.virtueai.com/2025/08/22/mcpguard-first-agent-based-mcp-scanner-to-protect-ai-agents/